CDI College

BDA400 - Data Science Tools and Techniques

Assignment 6

Technical Analysis Using R - Visualization Stage

Student: Parvana Babali

Assignment Value: 15%

Repository Folder: TechnicalAnalysis / Assignment06

Repository Link: https://github.com/ParvanaB/TechnicalAnalysis/tree/main/Assignment06

LMS File: Babali_Parvana_CA_BDA400_A06.pdf

Interactive R Shiny dashboard for portfolio visualization, technical indicators, and trading signals.

1. Project Overview

This project implements the visualization stage of the TechnicalAnalysis portfolio project. The application is an interactive R Shiny dashboard that retrieves historical stock data from Yahoo Finance, lets the user select dates, time frames, and chart types, displays technical indicators, and generates Buy, Sell, or Hold trading signals.

2. Repository and Submission

Direct Assignment 6 repository link:

https://github.com/ParvanaB/TechnicalAnalysis/tree/main/Assignment06

The repository folder contains app.R, setup_packages.R, portfolio.txt, README.md, the report, and a validation checklist.

3. Data Collection and Setup

Yahoo Finance is used as the financial-data source through quantmod::getSymbols(). The application accepts a stock symbol and date range, retrieves OHLCV data, handles invalid symbols with tryCatch(), and converts the returned xts data into a standard data frame for visualization.

```
required_packages <- c("shiny", "ggplot2", "quantmod", "TTR")

library(shiny)
library(ggplot2)
library(quantmod)
library(TTR)

stock_data <- getSymbols(
  symbol,
  src = "yahoo",
  from = start_date,
  to = end_date,
  auto.assign = FALSE
)
```

4. Interactive Dashboard Controls

- Stock symbol: select a portfolio symbol or type another valid Yahoo Finance ticker.
- Date range: choose the historical period to analyze.
- Time frame: Daily, Weekly, or Monthly.
- Chart type: Line, Area, or Candlestick.
- Technical indicators: independently toggle Moving Averages, RSI, and MACD.
- Moving averages: customize the short and long lookback periods.
- RSI: customize the RSI lookback period.
- Trading rule: choose MA Crossover or MA + RSI Confirmation.

5. Stock Data Visualization

The stock-price panel is reactive to all dashboard controls. The same OHLCV data can be viewed as a line, area, or candlestick chart. Weekly and monthly selections aggregate the Yahoo Finance data before indicators and trading signals are calculated.

```
selectInput(
  "chart_type",
  "Chart Type",
  choices = c("Line", "Area", "Candlestick")
)

selectInput(
  "time_frame",
  "Time Frame",
  choices = c("Daily", "Weekly", "Monthly")
)
```

6. Overlay Technical Indicators

Moving averages are drawn directly on the price chart. RSI and MACD are independently toggleable and appear in synchronized indicator panels. Separate panels preserve the correct numeric scale for oscillators while still allowing the user to turn each indicator on or off.

```
checkboxGroupInput(
  "technical_indicators",
  "Technical Indicators",
  choices = c("Moving Averages", "RSI", "MACD"),
  selected = c("Moving Averages", "RSI", "MACD")
)

df$SMA_Short <- SMA(df$Close, n = short_n)
df$SMA_Long  <- SMA(df$Close, n = long_n)
df$RSI       <- RSI(df$Close, n = rsi_n)

macd_values <- MACD(
  df$Close,
  nFast = 12,
  nSlow = 26,
  nSig = 9,
  maType = "EMA",
  percent = FALSE
)
```

7. Trading Rules and Chart Annotations

The main rule detects moving-average crossovers. A Buy signal occurs when the short moving average crosses above the long moving average; a Sell signal occurs when it crosses below. Otherwise the status is Hold. Users can optionally require RSI confirmation. Short/long MA periods and RSI confirmation thresholds are customizable from the dashboard.

```
cross_up <- (
  df$SMA_Short > df$SMA_Long &
  previous_short <= previous_long
)

cross_down <- (
```

```
  df$SMA_Short < df$SMA_Long &
  previous_short >= previous_long
)

df$Signal <- "Hold"
df$Signal[which(cross_up %in% TRUE)] <- "Buy"
df$Signal[which(cross_down %in% TRUE)] <- "Sell"
```

Buy events are marked on the price chart with upward markers and labels; Sell events are marked with downward markers and labels. Hold remains the default status when no new crossover occurs.

8. Error Handling and Validation

- Invalid or unavailable Yahoo Finance symbols are caught and displayed as a user-friendly message.
- The start date must be earlier than the end date.
- The short MA period must remain below the long MA period.
- The application checks that enough observations exist for the selected long MA period.
- Weekly/monthly aggregation is applied before technical indicators are calculated.
- The app displays a status block showing the source, symbol, time frame, date coverage, latest close, and latest signal.

9. How to Run the Dashboard

1. Open the Assignment06 folder in RStudio.
2. Run source("setup_packages.R") once to install missing packages.
3. Run shiny::runApp(), or open app.R and click Run App.
4. Select a symbol, dates, time frame, chart type, indicators, and trading parameters.
5. Use Refresh Data if you want to force a new Yahoo Finance request.

10. Grading-Criteria Coverage

Rubric Area	Implementation	Weight
Data Collection and Setup	Yahoo Finance, package setup, date/symbol controls, error handling	20%
Visualizing Stock Data	Shiny UI, interactive widgets, line/area/candlestick charts	30%
Overlay Technical Indicators	MA, RSI, MACD with independent on/off controls	25%
Trading Rules and Annotations	Customizable crossover rules, Buy/Sell/Hold, chart labels	25%

11. Repository File Structure

```
TechnicalAnalysis/
└── Assignment06/
    ├── app.R
```

```
├── setup_packages.R
├── portfolio.txt
├── README.md
├── validation_checklist.txt
├── Babali_Parvana_CA_BDA400_A06.pdf
└── Babali_Parvana_CA_BDA400_A06.docx
```

12. Key Implementation Notes

The dashboard intentionally displays RSI and MACD in separate panels rather than forcing them onto the stock-price axis. Moving averages share the stock-price scale and are therefore overlaid directly on the main chart. This keeps the visualization readable while still providing dynamic indicator toggling.

The portfolio file contains AAPL, MSFT, GOOGL, AMZN, and NVDA. The selectize input also allows the user to type another Yahoo Finance symbol, so the dashboard is not limited to those five companies.

Appendix A - Complete app.R

```
# BDA400 Assignment 6 - Technical Analysis Using R, Visualization Stage
# Student: Parvana Babali
# Repository: TechnicalAnalysis/Assignment06
#
# Dashboard features:
# - Yahoo Finance data via quantmod
# - Daily, weekly, and monthly time frames
# - Line, area, and candlestick price charts
# - Toggleable Moving Average, RSI, and MACD indicators
# - Customizable MA and RSI parameters
# - Buy/Sell/Hold rules with chart annotations
# - Error handling for missing/invalid symbols and insufficient data

required_packages <- c("shiny", "ggplot2", "quantmod", "TTR")

missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]

if (length(missing_packages) > 0) {
  stop(
    paste0(
      "Missing package(s): ",
      paste(missing_packages, collapse = ", "),
      ". Run setup_packages.R first."
    )
  )
}

library(shiny)
library(ggplot2)
library(quantmod)
library(TTR)

# ------------------------------------------------------------
# Portfolio configuration
# ------------------------------------------------------------

if (file.exists("portfolio.txt")) {
```

```
  portfolio_symbols <- scan(
    "portfolio.txt",
    what = "character",
    quiet = TRUE
  )
  portfolio_symbols <- unique(toupper(trimws(portfolio_symbols)))
  portfolio_symbols <- portfolio_symbols[nzchar(portfolio_symbols)]
} else {
  portfolio_symbols <- c("AAPL", "MSFT", "GOOGL", "AMZN", "NVDA")
}

if (length(portfolio_symbols) == 0) {
  portfolio_symbols <- c("AAPL", "MSFT", "GOOGL", "AMZN", "NVDA")
}

# ------------------------------------------------------------
# Helper functions
# ------------------------------------------------------------

fetch_stock_data <- function(symbol, from, to) {
  symbol <- toupper(trimws(symbol))

  if (!nzchar(symbol)) {
    stop("Please enter a stock symbol.")
  }

  result <- tryCatch(
    {
      suppressWarnings(
        getSymbols(
          Symbols = symbol,
          src = "yahoo",
          from = as.Date(from),
          to = as.Date(to) + 1,
          auto.assign = FALSE
        )
      )
    },
    error = function(e) {
      stop(
        paste0(
          "Unable to download data for ",
          symbol,
          ". Check the symbol, internet connection, and selected dates."
        )
      )
    }
  )

  if (NROW(result) == 0) {
    stop("Yahoo Finance returned no observations for the selected period.")
  }

  result
}

aggregate_time_frame <- function(stock_xts, time_frame) {
  if (time_frame == "Weekly") {
    return(to.weekly(stock_xts, indexAt = "endof", drop.time = TRUE))
```

```
  }

  if (time_frame == "Monthly") {
    return(to.monthly(stock_xts, indexAt = "endof", drop.time = TRUE))
  }

  stock_xts
}

xts_to_ohlcv <- function(stock_xts) {
  data.frame(
    Date = as.Date(index(stock_xts)),
    Open = as.numeric(Op(stock_xts)),
    High = as.numeric(Hi(stock_xts)),
    Low = as.numeric(Lo(stock_xts)),
    Close = as.numeric(Cl(stock_xts)),
    Volume = as.numeric(Vo(stock_xts)),
    stringsAsFactors = FALSE
  )
}

calculate_indicators <- function(df, short_n, long_n, rsi_n) {
  if (short_n >= long_n) {
    stop("Short moving-average period must be smaller than the long period.")
  }

  if (nrow(df) < long_n) {
    stop(
      paste0(
        "Not enough observations for the selected long MA period (",
        long_n,
        "). Choose a longer date range or a smaller period."
      )
    )
  }

  df$SMA_Short <- SMA(df$Close, n = short_n)
  df$SMA_Long <- SMA(df$Close, n = long_n)
  df$RSI <- RSI(df$Close, n = rsi_n)

  macd_values <- MACD(
    df$Close,
    nFast = 12,
    nSlow = 26,
    nSig = 9,
    maType = "EMA",
    percent = FALSE
  )

  df$MACD <- as.numeric(macd_values[, "macd"])
  df$MACD_Signal <- as.numeric(macd_values[, "signal"])
  df$MACD_Histogram <- df$MACD - df$MACD_Signal

  df
}

generate_signals <- function(df, rule, rsi_buy, rsi_sell) {
  previous_short <- c(NA, head(df$SMA_Short, -1))
  previous_long <- c(NA, head(df$SMA_Long, -1))
```

```
  cross_up <- (
    df$SMA_Short > df$SMA_Long &
      previous_short <= previous_long
  )

  cross_down <- (
    df$SMA_Short < df$SMA_Long &
      previous_short >= previous_long
  )

  if (rule == "MA + RSI Confirmation") {
    buy_condition <- cross_up & !is.na(df$RSI) & df$RSI <= rsi_buy
    sell_condition <- cross_down & !is.na(df$RSI) & df$RSI >= rsi_sell
  } else {
    buy_condition <- cross_up
    sell_condition <- cross_down
  }

  df$Signal <- "Hold"
  df$Signal[which(buy_condition %in% TRUE)] <- "Buy"
  df$Signal[which(sell_condition %in% TRUE)] <- "Sell"

  df
}

latest_non_hold_signal <- function(df) {
  signal_rows <- df[df$Signal %in% c("Buy", "Sell"), , drop = FALSE]

  if (nrow(signal_rows) == 0) {
    return("Hold - no crossover signal in the selected period")
  }

  last_row <- tail(signal_rows, 1)
  paste0(
    last_row$Signal,
    " on ",
    format(last_row$Date, "%Y-%m-%d")
  )
}

# ------------------------------------------------------------
# User interface
# ------------------------------------------------------------

ui <- fluidPage(
  tags$head(
    tags$style(HTML("
      body { background-color: #f6f8fb; }
      .dashboard-title { font-weight: 700; margin-bottom: 5px; }
      .dashboard-subtitle { color: #5f6b7a; margin-bottom: 20px; }
      .well { background-color: white; border-radius: 8px; }
      .panel-card {
        background: white;
        border: 1px solid #e4e8ef;
        border-radius: 8px;
        padding: 15px;
        margin-bottom: 15px;
      }
```

```
  "))
),

div(
  class = "dashboard-title",
  h2("Technical Analysis Portfolio Dashboard")
),
div(
  class = "dashboard-subtitle",
  "Yahoo Finance data with interactive indicators and trading signals"
),

sidebarLayout(
  sidebarPanel(
    width = 3,

    selectizeInput(
      "symbol",
      "Stock Symbol",
      choices = portfolio_symbols,
      selected = portfolio_symbols[1],
      options = list(create = TRUE)
    ),

    dateRangeInput(
      "date_range",
      "Date Range",
      start = Sys.Date() - (365 * 5),
      end = Sys.Date()
    ),

    selectInput(
      "time_frame",
      "Time Frame",
      choices = c("Daily", "Weekly", "Monthly"),
      selected = "Daily"
    ),

    selectInput(
      "chart_type",
      "Chart Type",
      choices = c("Line", "Area", "Candlestick"),
      selected = "Line"
    ),

    checkboxGroupInput(
      "technical_indicators",
      "Technical Indicators",
      choices = c("Moving Averages", "RSI", "MACD"),
      selected = c("Moving Averages", "RSI", "MACD")
    ),

    tags$hr(),

    sliderInput(
      "short_ma",
      "Short Moving Average",
      min = 5,
      max = 40,
```

```
    value = 20,
    step = 1
  ),

  sliderInput(
    "long_ma",
    "Long Moving Average",
    min = 20,
    max = 100,
    value = 50,
    step = 1
  ),

  sliderInput(
    "rsi_period",
    "RSI Period",
    min = 5,
    max = 30,
    value = 14,
    step = 1
  ),

  selectInput(
    "signal_rule",
    "Trading Rule",
    choices = c(
      "MA Crossover",
      "MA + RSI Confirmation"
    ),
    selected = "MA Crossover"
  ),

  conditionalPanel(
    condition = "input.signal_rule == 'MA + RSI Confirmation'",
    sliderInput(
      "rsi_buy",
      "Buy RSI Maximum",
      min = 20,
      max = 50,
      value = 40,
      step = 1
    ),
    sliderInput(
      "rsi_sell",
      "Sell RSI Minimum",
      min = 50,
      max = 80,
      value = 60,
      step = 1
    )
  ),

  actionButton(
    "refresh",
    "Refresh Data",
    class = "btn-primary"
  )
),
```

```
  mainPanel(
    width = 9,

    div(
      class = "panel-card",
      h4("Dashboard Status"),
      verbatimTextOutput("status_text")
    ),

    div(
      class = "panel-card",
      h4("Stock Price and Trading Signals"),
      plotOutput("price_chart", height = "480px")
    ),

    conditionalPanel(
      condition = "input.technical_indicators.indexOf('RSI') >= 0",
      div(
        class = "panel-card",
        h4("Relative Strength Index (RSI)"),
        plotOutput("rsi_chart", height = "260px")
      )
    ),

    conditionalPanel(
      condition = "input.technical_indicators.indexOf('MACD') >= 0",
      div(
        class = "panel-card",
        h4("Moving Average Convergence Divergence (MACD)"),
        plotOutput("macd_chart", height = "280px")
      )
    ),

    fluidRow(
      column(
        6,
        div(
          class = "panel-card",
          h4("Summary"),
          tableOutput("summary_table")
        )
      ),
      column(
        6,
        div(
          class = "panel-card",
          h4("Recent Buy/Sell Signals"),
          tableOutput("signal_table")
        )
      )
    ),

    div(
      class = "panel-card",
      h4("Latest Observations"),
      tableOutput("data_preview")
    )
  )
)
```

```
)

# ------------------------------------------------------------
# Server
# ------------------------------------------------------------

server <- function(input, output, session) {

  # Re-fetch data when the user changes the symbol/date range or
  # presses Refresh Data.
  raw_stock <- reactive({
    input$refresh

    validate(
      need(
        input$date_range[1] < input$date_range[2],
        "The start date must be earlier than the end date."
      )
    )

    tryCatch(
      fetch_stock_data(
        input$symbol,
        input$date_range[1],
        input$date_range[2]
      ),
      error = function(e) {
        validate(need(FALSE, e$message))
      }
    )
  })

  stock_data <- reactive({
    x <- raw_stock()
    x <- aggregate_time_frame(x, input$time_frame)
    df <- xts_to_ohlcv(x)

    validate(
      need(
        nrow(df) > 0,
        "No data remains after applying the selected time frame."
      )
    )

    tryCatch(
      calculate_indicators(
        df,
        short_n = input$short_ma,
        long_n = input$long_ma,
        rsi_n = input$rsi_period
      ),
      error = function(e) {
        validate(need(FALSE, e$message))
      }
    )
  })

  analysis_data <- reactive({
    df <- stock_data()
```

```
  rsi_buy_value <- if (is.null(input$rsi_buy)) 40 else input$rsi_buy
  rsi_sell_value <- if (is.null(input$rsi_sell)) 60 else input$rsi_sell

  generate_signals(
    df,
    rule = input$signal_rule,
    rsi_buy = rsi_buy_value,
    rsi_sell = rsi_sell_value
  )
})

output$status_text <- renderText({
  df <- analysis_data()

  paste(
    "Data source: Yahoo Finance",
    paste0("Symbol: ", toupper(input$symbol)),
    paste0("Time frame: ", input$time_frame),
    paste0("Observations: ", nrow(df)),
    paste0(
      "Coverage: ",
      min(df$Date),
      " to ",
      max(df$Date)
    ),
    paste0(
      "Latest close: $",
      format(
        round(tail(df$Close, 1), 2),
        nsmall = 2
      )
    ),
    paste0(
      "Latest trading status: ",
      latest_non_hold_signal(df)
    ),
    sep = "\n"
  )
})

output$price_chart <- renderPlot({
  df <- analysis_data()

  p <- ggplot(df, aes(x = Date))

  if (input$chart_type == "Line") {
    p <- p +
      geom_line(
        aes(y = Close),
        linewidth = 0.8
      )
  }

  if (input$chart_type == "Area") {
    p <- p +
      geom_area(
        aes(y = Close),
        alpha = 0.25
```

```
    ) +
    geom_line(
      aes(y = Close),
      linewidth = 0.7
    )
}

if (input$chart_type == "Candlestick") {
  candle_width <- switch(
    input$time_frame,
    "Daily" = 0.35,
    "Weekly" = 2.0,
    "Monthly" = 8.0,
    0.35
  )

  p <- p +
    geom_segment(
      aes(
        x = Date,
        xend = Date,
        y = Low,
        yend = High
      ),
      linewidth = 0.45
    ) +
    geom_rect(
      aes(
        xmin = Date - candle_width,
        xmax = Date + candle_width,
        ymin = pmin(Open, Close),
        ymax = pmax(Open, Close),
        fill = Close >= Open
      ),
      alpha = 0.75
    ) +
    scale_fill_manual(
      values = c(
        "TRUE" = "#2f9e44",
        "FALSE" = "#e03131"
      ),
      guide = "none"
    )
}

if ("Moving Averages" %in% input$technical_indicators) {
  p <- p +
    geom_line(
      aes(y = SMA_Short),
      linewidth = 0.8,
      color = "#1971c2",
      na.rm = TRUE
    ) +
    geom_line(
      aes(y = SMA_Long),
      linewidth = 0.8,
      color = "#f08c00",
      na.rm = TRUE
    )
```

```
}

buy_rows <- df[df$Signal == "Buy", , drop = FALSE]
sell_rows <- df[df$Signal == "Sell", , drop = FALSE]

if (nrow(buy_rows) > 0) {
  p <- p +
    geom_point(
      data = buy_rows,
      aes(y = Close),
      color = "#2b8a3e",
      shape = 24,
      fill = "#2b8a3e",
      size = 3
    ) +
    geom_text(
      data = buy_rows,
      aes(y = Close, label = "Buy"),
      color = "#2b8a3e",
      vjust = -1.1,
      size = 3
    )
}

if (nrow(sell_rows) > 0) {
  p <- p +
    geom_point(
      data = sell_rows,
      aes(y = Close),
      color = "#c92a2a",
      shape = 25,
      fill = "#c92a2a",
      size = 3
    ) +
    geom_text(
      data = sell_rows,
      aes(y = Close, label = "Sell"),
      color = "#c92a2a",
      vjust = 1.7,
      size = 3
    )
}

p +
  labs(
    title = paste(
      toupper(input$symbol),
      "-",
      input$chart_type,
      "Chart"
    ),
    subtitle = paste(
      input$time_frame,
      "data |",
      input$signal_rule
    ),
    x = "Date",
    y = "Price (USD)",
    caption = paste0(
```

```
      "MA settings: ",
      input$short_ma,
      " / ",
      input$long_ma
    )
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )
})

output$rsi_chart <- renderPlot({
 req("RSI" %in% input$technical_indicators)
 df <- analysis_data()

 ggplot(df, aes(x = Date, y = RSI)) +
  geom_line(
    linewidth = 0.75,
    na.rm = TRUE
  ) +
  geom_hline(
    yintercept = 70,
    linetype = "dashed"
  ) +
  geom_hline(
    yintercept = 30,
    linetype = "dashed"
  ) +
  annotate(
    "text",
    x = max(df$Date, na.rm = TRUE),
    y = 72,
    label = "Overbought",
    hjust = 1,
    size = 3
  ) +
  annotate(
    "text",
    x = max(df$Date, na.rm = TRUE),
    y = 28,
    label = "Oversold",
    hjust = 1,
    size = 3
  ) +
  coord_cartesian(ylim = c(0, 100)) +
  labs(
    x = "Date",
    y = "RSI",
    title = paste0(
      "RSI (",
      input$rsi_period,
      "-period)"
    )
  ) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank())
})
```

```
output$macd_chart <- renderPlot({
  req("MACD" %in% input$technical_indicators)
  df <- analysis_data()

  ggplot(df, aes(x = Date)) +
    geom_col(
      aes(y = MACD_Histogram),
      alpha = 0.35,
      na.rm = TRUE
    ) +
    geom_line(
      aes(y = MACD),
      linewidth = 0.8,
      na.rm = TRUE
    ) +
    geom_line(
      aes(y = MACD_Signal),
      linewidth = 0.8,
      linetype = "dashed",
      na.rm = TRUE
    ) +
    geom_hline(
      yintercept = 0,
      linewidth = 0.4
    ) +
    labs(
      x = "Date",
      y = "MACD",
      title = "MACD (12, 26, 9)"
    ) +
    theme_minimal(base_size = 11) +
    theme(panel.grid.minor = element_blank())
})

output$summary_table <- renderTable({
  df <- analysis_data()

  data.frame(
    Metric = c(
      "Symbol",
      "Latest Close",
      "Minimum Close",
      "Maximum Close",
      "Average Close",
      "Current RSI",
      "Current Signal"
    ),
    Value = c(
      toupper(input$symbol),
      paste0("$", round(tail(df$Close, 1), 2)),
      paste0("$", round(min(df$Close, na.rm = TRUE), 2)),
      paste0("$", round(max(df$Close, na.rm = TRUE), 2)),
      paste0("$", round(mean(df$Close, na.rm = TRUE), 2)),
      round(tail(na.omit(df$RSI), 1), 2),
      tail(df$Signal, 1)
    ),
    stringsAsFactors = FALSE
  )
```

```
  }, striped = TRUE, bordered = TRUE)

  output$signal_table <- renderTable({
    df <- analysis_data()
    signals <- df[df$Signal %in% c("Buy", "Sell"), c(
      "Date",
      "Close",
      "Signal"
    ), drop = FALSE]

    if (nrow(signals) == 0) {
      return(
        data.frame(
          Message = "No Buy/Sell crossover signals in the selected period."
        )
      )
    }

    tail(signals, 10)
  }, striped = TRUE, bordered = TRUE)

  output$data_preview <- renderTable({
    df <- analysis_data()

    preview <- tail(
      df[, c(
        "Date",
        "Open",
        "High",
        "Low",
        "Close",
        "Volume",
        "SMA_Short",
        "SMA_Long",
        "RSI",
        "Signal"
      )],
      10
    )

    numeric_columns <- sapply(preview, is.numeric)
    preview[numeric_columns] <- lapply(
      preview[numeric_columns],
      function(x) round(x, 2)
    )

    preview
  }, striped = TRUE, bordered = TRUE)
}

shinyApp(ui = ui, server = server)
```